

Large Language Models

Naeemullah Khan

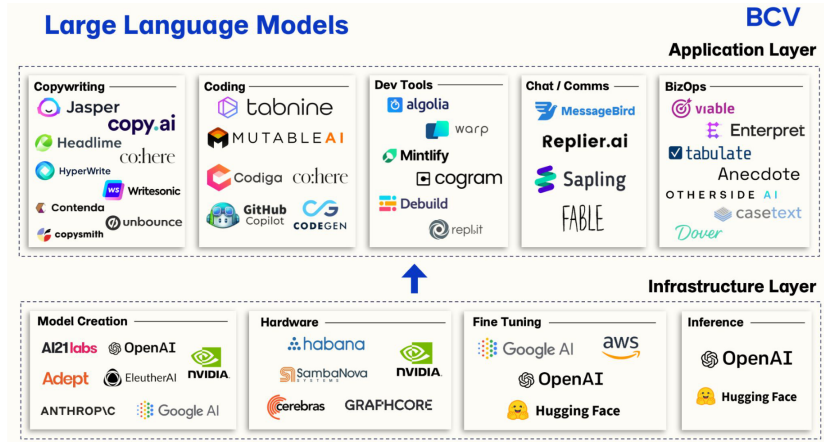
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026



Source: R. Garg and S. Crowder, "Large Language Models Will Redefine B2B Software", Bain Capital Ventures, 2022.

1. Motivation
2. Learning Outcomes
3. BERT and GPT Concepts
4. Scaling Laws for LLMs
5. Pre-training Overview
6. Tokenization: Why It Matters?
7. Level of Tokenization
8. Byte-Pair Encoding (BPE)
9. Token-Free LLMs
10. Tokenization Pitfalls
11. Context Window Problems
12. Moving Towards Larger Context Windows
13. Limitations of LLMs
14. Summary
15. Future Directions

► Why LLMs?

- Transforming NLP: ChatGPT, Claude, Gemini, etc.
- Achieving human-like generation and comprehension.
- Central to tasks like summarization, translation, reasoning.

► Need for deeper understanding:

- LLMs are expensive to train and operate.
- Design decisions impact performance significantly (e.g., tokenization, scaling laws).

By the end of this session, you should be able to:

- ▶ Explain core architectures like BERT and GPT.
- ▶ Understand scaling laws for LLM development.
- ▶ Describe tokenization strategies and their impact.
- ▶ Discuss limitations of current models (e.g., context window).
- ▶ Explore emerging directions like token-free LLMs.

► BERT (Bidirectional Encoder Representations from Transformers)

- Uses transformer encoder only.
- Trained with Masked Language Modeling (MLM).
- **Bidirectional:** Considers both left and right context.
- Fine-tuned for tasks like QA, classification.
- **Architecture:**
 - Layers of encoder blocks.
 - Positional encodings added to embeddings.
 - Self-attention heads capture dependencies.

- ▶ One of the biggest challenges in LM-building used to be the lack of task-specific training data.
- ▶ What if we learn an effective representation that can be applied to a variety of downstream tasks?
 - Word2vec (2013)
 - GloVe (2014)

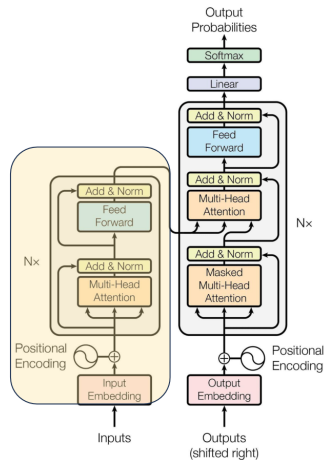


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

BERT Pre-Training Corpus:

- ▶ English Wikipedia – 2,500 million words
- ▶ Book Corpus – 800 million words

BERT Pre-Training Tasks:

- ▶ MLM (Masked Language Modeling)
- ▶ NSP (Next Sentence Prediction)

BERT Pre-Training Results:

- ▶ BERT-Base – 110M Params
- ▶ BERT-Large – 340M Params

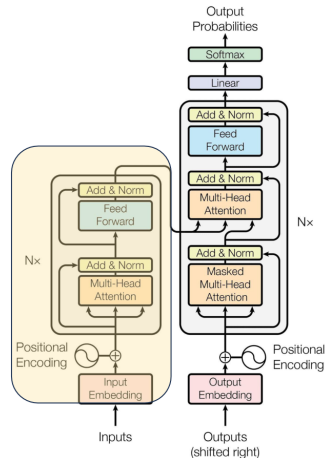
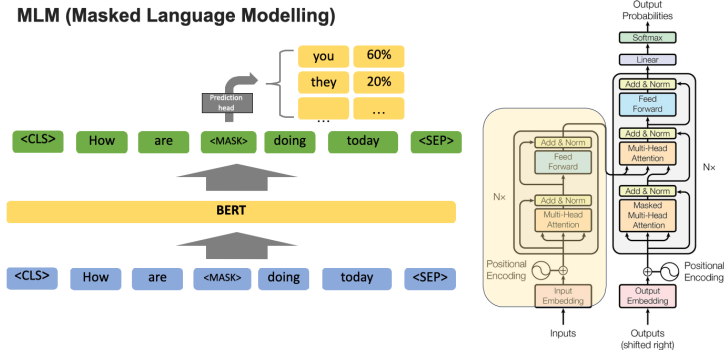
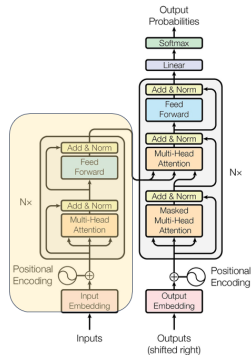
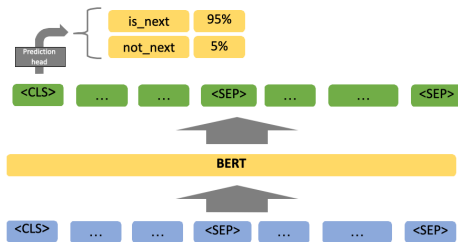


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 149; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.

NSP (Next Sentence Prediction)



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 150; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.

BERT Fine-Tuning:

- ▶ Simply add a task-specific module after the last encoder layer to map it to the desired dimension.
- ▶ **Classification Tasks:**
 - Every input starts with a special [CLS] token; its final encoder output acts as a summary of the whole sequence.
 - Add a feed-forward layer on top of that output to predict the class.
- ▶ **Question Answering Tasks:**
 - Train two extra vectors to mark the beginning and end of the answer from the paragraph.
- ▶ ...

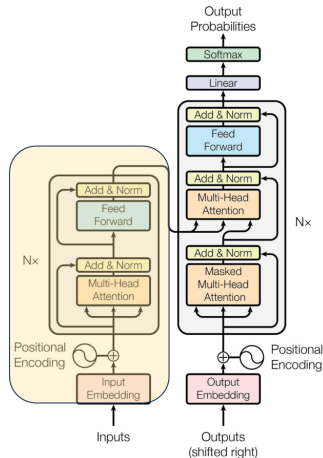


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

BERT Evaluation:

- ▶ **General Language Understanding Evaluation (GLUE):**
 - Sentence pair tasks
 - Single sentence classification
- ▶ **Stanford Question Answering Dataset (SQuAD)**

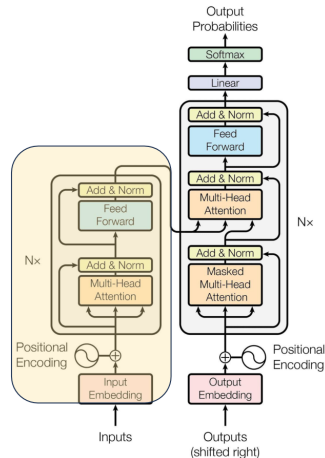


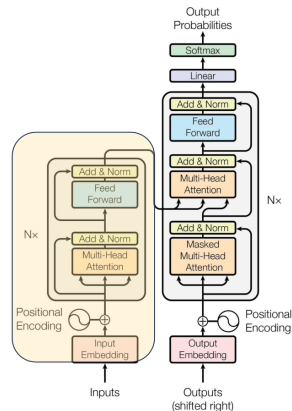
Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

BERT Evaluation:

System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	Average
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

System	Dev EM	Dev F1	Test EM	Test F1
Leaderboard (Oct 8th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
#1 Single - nlnet	-	-	83.5	90.1
#2 Single - QANet	-	-	82.5	89.3
Published				
BiDAF+ELMo (Single)	-	85.8	-	-
R.M. Reader (Single)	78.9	86.3	79.5	86.6
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

Table 2: SQuAD results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.



Source: Devlin et al., "BERT", NAACL 2019, Tables 1–2; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.

► GPT (Generative Pre-trained Transformer)

- Uses transformer **decoder only**.
- Trained with next-token prediction.
- **Unidirectional**: Considers left context only.
- Fine-tuned for text generation, dialogue systems.
- **Architecture**:
 - Layers of decoder blocks.
 - Causal masking to prevent future token access.
 - Self-attention heads capture sequential dependencies.

- ▶ Similarly motivated as BERT, though differently designed
- ▶ Can we use large amounts of unlabeled data to pretrain an LM that understands general patterns?

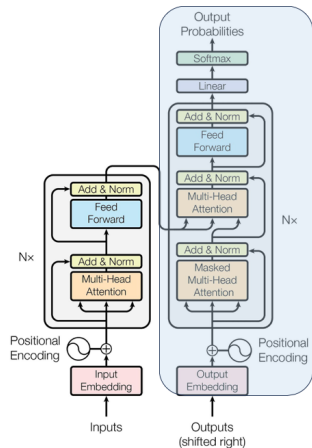
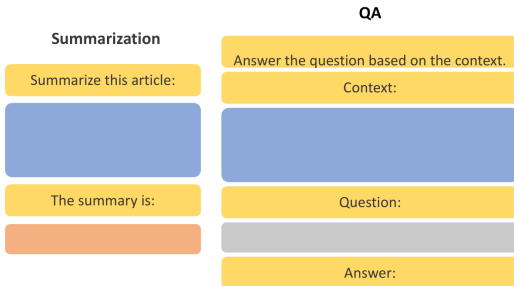


Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

GPT Fine-Tuning:

- Prompt-format task-specific text as a continuous stream for the model to fit



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 160.

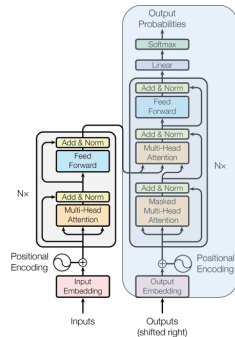


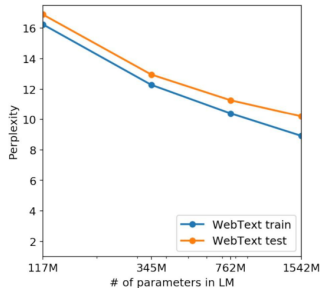
Diagram: Vaswani et al., NeurIPS 2017, Fig. 1.

► Key Differences:

Feature	BERT	GPT
Directionality	Bidirectional	Unidirectional
Objective	Masked Language Modeling (MLM)	Causal Language Modeling (CLM)
Output	Contextual embeddings	Text generation
Usage	Downstream tasks (e.g., classification, QA)	Generation, few-shot learning

- ▶ **Kaplan et al. (2020):** “Scaling Laws for Neural Language Models”
- ▶ Performance improves predictably with:
 - More parameters
 - More compute
 - Larger datasets
- ▶ **Optimal allocation of compute:** Train bigger models with less data, rather than small models with lots of data.
- ▶ **Implication:** LLMs like GPT-3 (175B), GPT-4 (est. >500B) are products of scaling laws.

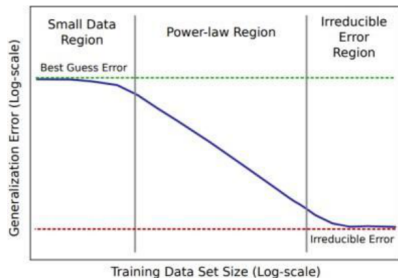
- Scaling improves the perplexity of the LM and improves performance



Source: Radford et al., "Language Models are Unsupervised Multitask Learners" (GPT-2), OpenAI 2019, Fig. 4.

Why is this interesting? Look at data scaling

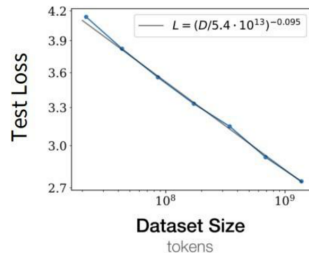
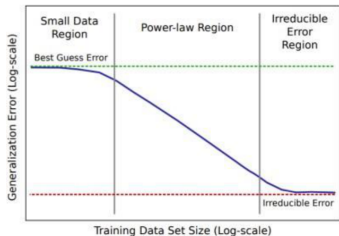
- We know that typical scaling effects look like this when we increase the amount of training data



Source: Hestness et al., “Deep Learning Scaling is Predictable, Empirically”, 2017, Fig. 6.

Why is this interesting? Look at data scaling

- ▶ Loss and dataset size is linear on a log-log plot
- ▶ This is “power-law scaling”



Source: Hestness et al., 2017, Fig. 6 (left); Kaplan et al., “Scaling Laws for Neural Language Models”, 2020, Fig. 1 (right).

- ▶ Can we understand scaling by positing scaling laws?
- ▶ With scaling laws, we can make decisions on architecture, data, and hyperparameters by training smaller models.
- ▶ **OpenAI Study:** *Scaling Laws for Neural Language Models* (Kaplan et al., 2020)

► **OpenAI Study:** Scaling Laws for Neural Language Models (Kaplan et al., 2020)

Key Findings:

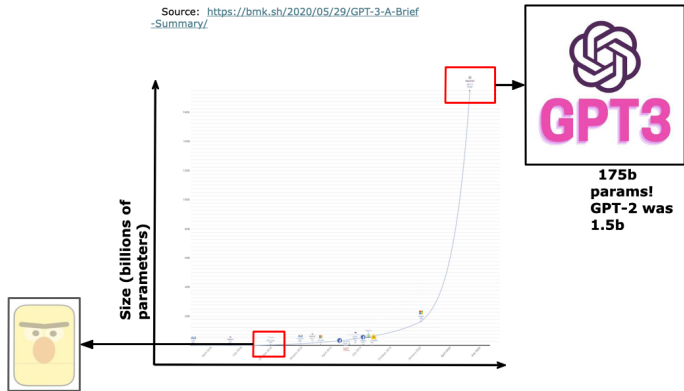
- Performance depends strongly on scale, and only weakly on the model shape.
- Larger models are more sample-efficient.
- Smooth power laws ($y = ax^b$) observed between empirical performance and:
 - N – number of parameters
 - D – dataset size
 - C – compute

- ▶ Kaplan's split favours parameters: $10\times$ the compute buys $5.5\times$ the parameters but only $1.8\times$ the tokens.
- ▶ **DeepMind Study:** [Training Compute-Optimal Large Language Models \(Hoffmann et al., 2022\)](#) refit the laws over 400+ models (70M–16B parameters, 5B–500B tokens).
- ▶ **Result:** N and D should scale *equally*: double the model, double the tokens, or about 20 training tokens per parameter.
- ▶ **Chinchilla vs. Gopher**, at the same compute budget:
 - Gopher: 280B parameters, 300B tokens
 - Chinchilla: 70B parameters, 1.4T tokens, and it outperforms Gopher
- ▶ **Takeaway:** those models were undertrained, not too small. A scaling law is an empirical fit, and can be refit when the experiment is run more carefully.

- ▶ The effect of some hyperparameters on large language models (LMs) can be predicted before full training: optimizer choice (Adam vs. SGD), model depth, and architecture (LSTM vs. Transformer).

Idea:

- Train a few smaller models
- Establish a scaling law (e.g., Adam vs. SGD scaling law)
- Select optimal hyperparameters based on the scaling law prediction



Source: Leo Gao, "Why GPT-3 Matters", 2020 (chart edited by WillStats).

► Emergent abilities:

- Not present in smaller models but is present in larger models
- Do LLMs like GPT-3 have these?

► Findings:

- GPT-3 trained on text can do arithmetic problems like addition and subtraction
- Different abilities “emerge” at different scales

► Emergent abilities:

- Not present in smaller models but is present in larger models
- Do LLMs like GPT-3 have these?

► Findings:

- GPT-3 trained on text can do arithmetic problems like addition and subtraction
- Different abilities “emerge” at different scales
- **Model scale is not the only contributor to emergence** – for 14 BIG-Bench tasks, LaMDA 137B and GPT-3 175B models perform at near-random, but PaLM 62B achieves above-random performance
- Problems LLMs can’t solve today may be emergent for future LLMs

► Pre-training Phase:

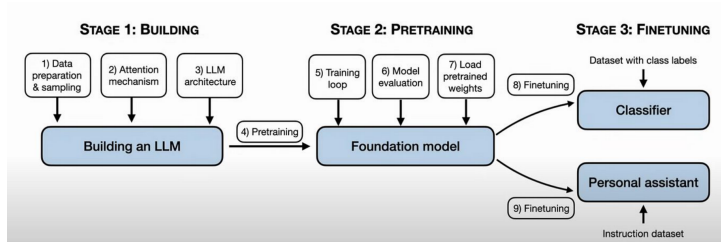
- Large-scale unsupervised training on corpus (e.g., Common Crawl, Books)
- Objective: learn general-purpose language representations

► Why Pre-train?

- Data-efficient fine-tuning
- Enables zero-shot and few-shot capabilities
- Foundation for instruction tuning, alignment

► Challenges:

- Massive compute costs
- Environmental concerns (carbon footprint)



Source: S. Raschka, "Build a Large Language Model (From Scratch)", Manning, 2024.

- ▶ **Auto-regressive Pre-training**

Train to predict the next token on very large scale corpora (~ 2 trillion tokens)

- ▶ **Instruction Fine-tuning / Supervised Fine-tuning (SFT)**

Fine-tune the pre-trained model with pairs of (instruction + input, output) using a large dataset, then further with a small high-quality dataset

Instruction fine-tuning provides, as a prefix, a natural language description of the task along with the input.

- ▶ **Example:** Translate into French this sentence: my name is $\rightarrow$ je m'appelle

► Objective function

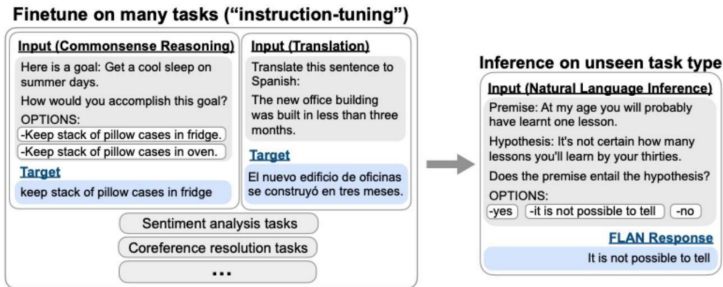
- Loss computed only for target tokens in SFT, all tokens are targets in pre-training

► Input and Target

- Instruction + input as input with the target in SFT and only input as input with shifted input as target

► Purpose

- Pre-training makes good generalist auto-completes but good SFT builds models that can do many unseen tasks
- SFT can also guide nature of outputs in terms of safety and helpfulness



Source: Wei et al., “Finetuned Language Models Are Zero-Shot Learners” (FLAN), ICLR 2022, Fig. 1.

- ▶ LLMs may produce
 - Harmful text – unparliamentary language, bias and discrimination
 - Text that can cause direct harm – allowing easy access to dangerous information
- ▶ Therefore, LLMs should be trained to produce outputs that align with human preferences and values
- ▶ Modern LLMs do so by using SFT and by using human preference directly in model training

▶ Language $\neq$ Input

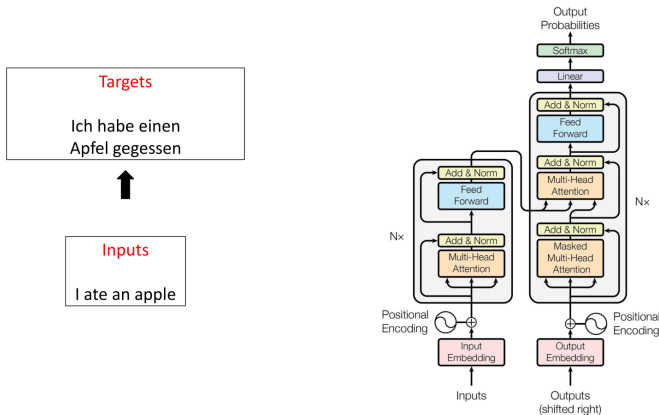
- Text must be converted to numbers $\rightarrow$ tokens.

▶ Tokenization:

- Splits text into manageable units.
- Balances granularity and vocabulary size.

▶ Good tokenizer =

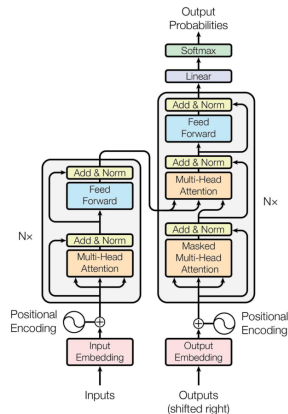
- Efficient sequence length
- High vocabulary coverage
- Robust across domains (code, multilingual, etc.)



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 7; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.

Processing Inputs

Inputs
I ate an apple

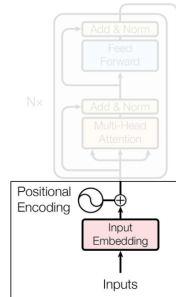


Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 8; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.

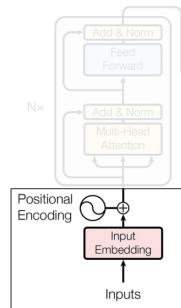
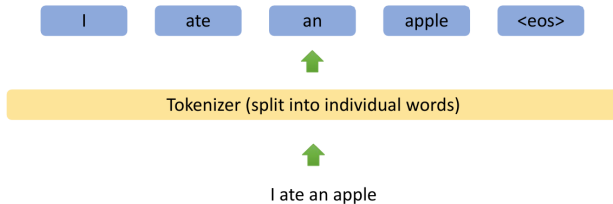
Tokenizer (split into individual words)



I ate an apple



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 9; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.



Source: CMU 11-785 (Spring 2024), Lecture 19: Transformers and Newer Architectures, slide 10; architecture panel: Vaswani et al., NeurIPS 2017, Fig. 1.

▶ Character-level

- Fine-grained, handles unknowns
- Long sequences, slow training

▶ Word-level

- Intuitive, natural boundaries
- Large vocab, OOV (Out-of-Vocab) issues

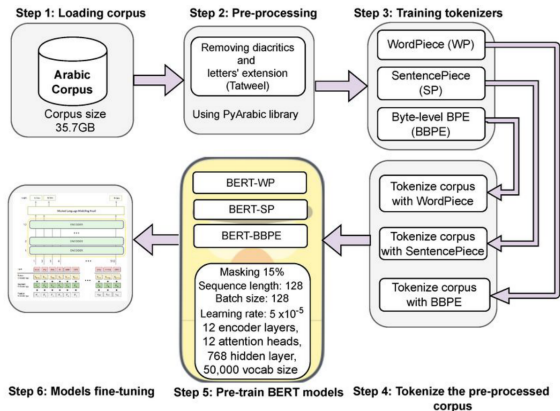
▶ Subword-level

- Balance of generalization & compactness
- Requires segmentation algorithm

▶ How subword vocabularies are built

- BPE, WordPiece, Unigram LM: all learned from the corpus
- SentencePiece applies them to raw text, with no pre-tokenizer

Different Levels of Tokenization



Source: Qarah and Alsanoosy, "A Comprehensive Analysis of Various Tokenizers for Arabic Large Language Models", Applied Sciences 14(13), 2024, Fig. 1.

Types Of Tokenization

"Machine",
"learning",
"is", "fun", "."

Word-Based

"ma",
"chine",
"learn", "ing"

Subword-Based

"M", "a", "c",
"h", "i", "n",
"e", "l", "e",
"a", "r", "n",
"i", "n", "g"

Character-Based

Source: H. Bui, "Tokenization In NLP: Everything We Know So Far", Eastgate Software, 2024.

► How BPE works:

- Start with characters.
- Merge most frequent pair (e.g., “t”, “h” → “th”).
- Repeat until vocab size reached.

► Advantages:

- Handles unknown words well (e.g., unbelievably → un + believ + ably)
- Reduces sequence length

► Limitations:

- Not optimal for all scripts (e.g., Chinese)
- Word boundaries may be unclear

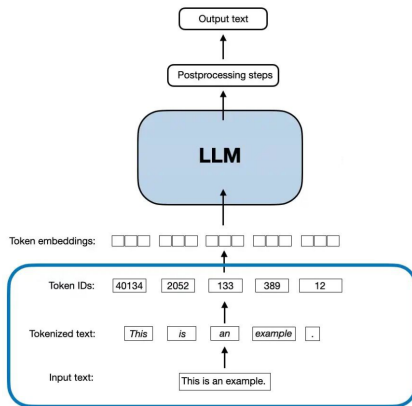
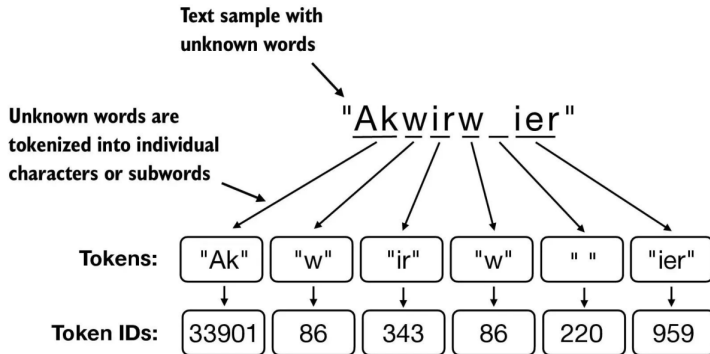
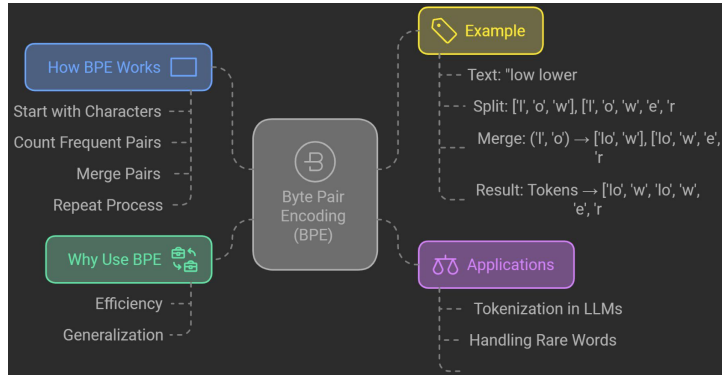


Figure adapted from S. Raschka, "Build a Large Language Model (From Scratch)", Manning, 2024, Ch. 2.



Source: S. Raschka, "Build a Large Language Model (From Scratch)", Manning, 2024, Ch. 2.

Byte-Pair Encoding (BPE)



Source: D. Alam and Vizura AI, "Understanding Byte Pair Encoding (BPE) in Large Language Models", Vizura's Substack, 2024.

► What are Token-Free LLMs?

- Models that operate directly on raw text without tokenization.
- Aim to eliminate the need for pre-defined tokens.

► Advantages:

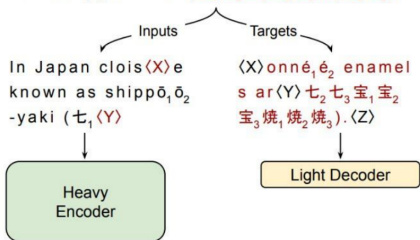
- Avoids tokenization errors and biases.
- Can handle arbitrary text lengths and formats.
- Potentially more efficient for certain tasks.

► Challenges:

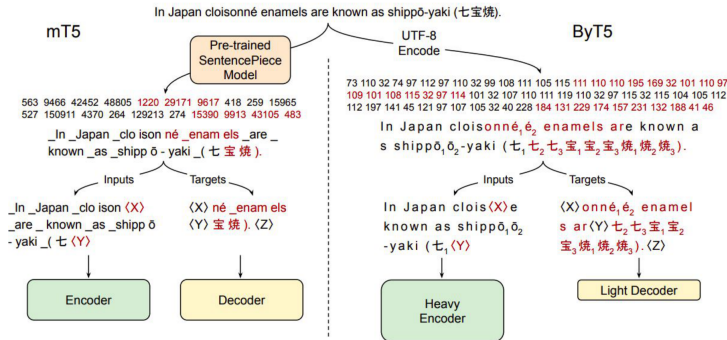
- Requires novel architectures to process raw text effectively.
- May struggle with long-range dependencies without tokenization.

ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models

s shippō₁ō₂-yaki (七₁七₂七₃宝₁宝₂宝₃焼₁焼₂焼₃).



Source: Xue et al., "ByT5", TACL 10, 2022, Fig. 1 (right).



Source: Xue et al., "ByT5", TACL 10, 2022, Fig. 1.

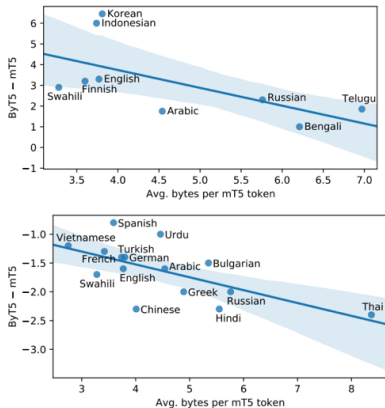


Figure 3: Per-language performance gaps between ByT5-Large and mT5-Large, as a function of each language’s “compression rate”. **Top:** TyDiQA-GoldP gap. **Bottom:** XNLI zero-shot gap.

Source: Xue et al., “ByT5”, TACL 10, 2022, Fig. 3.

► Example: ByT5 (Xue et al., 2022)

- Character-level variant of T5 that processes raw bytes instead of tokens.
- Eliminates the tokenizer, so any language or script is covered without changing the vocabulary.

► Pros:

- No preprocessing or tokenization pipeline required.
- Handles any language or script out of the box, with no unknown tokens.
- Wins below $\sim 1\text{B}$ parameters, on generative tasks, on spelling-sensitive word-level tasks, and on noisy input.

► Cons:

- Training is slower due to longer input sequences.
- Increased computational requirements.
- mT5 regains the advantage on classification above $\sim 1\text{B}$ parameters.

Source: Xue et al., “ByT5,” *TACL* 10, 2022, [arXiv:2105.13626](#).

► Current Research:

- Encoders that learn their own segmentation instead of using a fixed vocabulary: *CANINE*, *Charformer*.
- Decoders that group bytes into patches to keep sequences short: *MEGABYTE*, *Byte Latent Transformer*.

► Future Directions:

- Integrating token-free approaches with existing LLMs.
- Enhancing efficiency and scalability of token-free models.

- ▶ Tokenizers are trained on English-heavy corpora, so the same sentence costs very different numbers of tokens across languages.
- ▶ On parallel multilingual text, the shortest languages are already about 50% longer than English in tokens, and the worst cases are far larger: roughly $3\times$ for Arabic, $11\times$ for Burmese, and up to $15\times$ for scripts such as Shan.
- ▶ This is an **equity and cost problem**: longer token sequences mean higher API cost, more latency, and less usable context for speakers of under-represented languages, for the exact same content.
- ▶ Byte- and character-level models (previous section) reduce, but do not fully remove, the disparity.

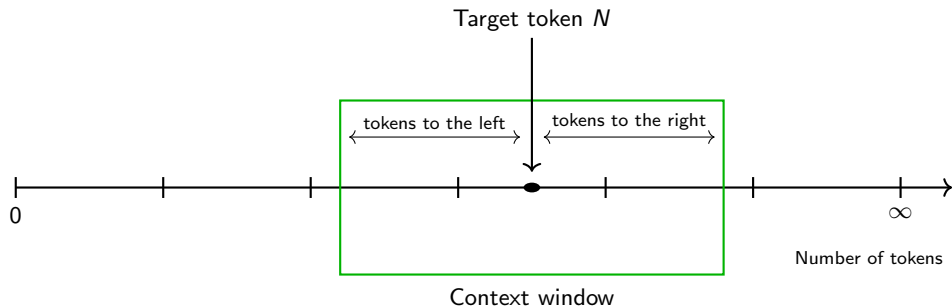
Source: Petrov, La Malfa, Torr, Bibi, “Language Model Tokenizers Introduce Unfairness Between Languages,” NeurIPS 2023, [arXiv:2305.15425](https://arxiv.org/abs/2305.15425).

- ▶ The tokenizer is trained first, then the model is trained on a (sometimes different) corpus. Tokens that live in the vocabulary but almost never appear in training end up with **untrained embeddings**.
- ▶ These **glitch tokens** trigger strange behavior: the model cannot repeat them, spells them wrong, or produces evasive or erratic output when they appear.
- ▶ The original examples were odd strings scraped into the GPT-2 vocabulary (the “SolidGoldMagikarp” family); similar artifacts appear as rare Arabic and Burmese tokens in multilingual tokenizers.
- ▶ Lesson: the tokenizer and the training data must stay in sync, and rare-token embeddings are a genuine failure surface.

Source: Rumbelow & Watkins, “SolidGoldMagikarp (plus, prompt generation),” LessWrong, 2023; Land & Bartolo, “Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models,” EMNLP 2024, [arXiv:2405.05417](https://arxiv.org/abs/2405.05417).

- ▶ Current LLMs have a finite “context window”:
 - **GPT-3:** 2K tokens
 - **GPT-4:** Up to 128K tokens
 - **Claude 3.5:** Up to 200K tokens
- ▶ **Problems:**
 - Long documents get truncated
 - Token limit affects reasoning
 - Difficult to do document-level QA or summarization

Example of a Context Window



Why **Bigger Context Windows** Aren't Always Better?

1

Information Overload

- Excess data can overwhelm the LLM.
- Reduces focus and slows performance.

2

Getting Lost in Data

- Large windows prioritize edges,
- Missing key middle info.

3

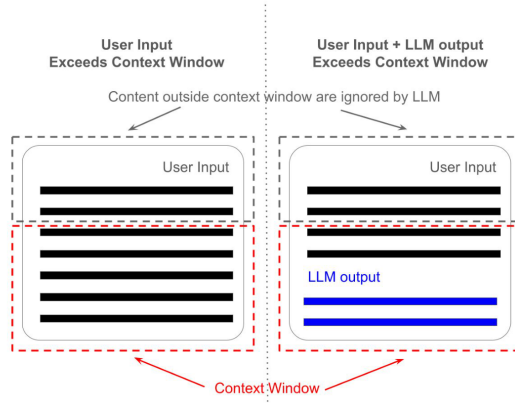
Poor Management

- More data leads to noise,
 - Redundancy, and potential bias.

4

Long-Range Issues

- Understanding relationships becomes harder with large context windows.



Source: "ChatGPT Context Window and Token Limit", 16x Prompt blog, 2024.

▶ **Efficient Attention Variants:**

- Longformer, BigBird, FlashAttention-2

▶ **Memory-Augmented Models:**

- Retrieval-augmented generation (RAG): fetch relevant passages and put them in the prompt (covered later)
- Memory layers (e.g., product-key memory)

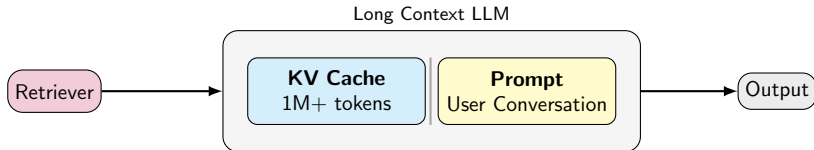
▶ **Chunking + Re-ranking:**

- Process in parts, summarize/join results

▶ **Recurrence or State Passing:**

- Transformer-XL, RWKV, RetNet: carry a fixed-size state forward instead of attending to every past token

- ▶ Serving a long context means keeping the **KV cache**: the keys and values already computed for every token so far, so that generating the next token does not recompute the whole prompt.
- ▶ The cache grows linearly with context length, so memory, not attention cost, is what usually caps the window in practice.



- ▶ High inference costs & latency
- ▶ Bias and toxic outputs
- ▶ Hallucination and factual inconsistency
- ▶ Limited interpretability
- ▶ Require vast pre-training data
- ▶ Poor domain generalization (medical, legal, etc.)

- ▶ **The training objective shapes the architecture.** Masked prediction over both directions gives BERT representations to fine-tune; next-token prediction over the left context gives GPT a generator.
- ▶ **Scaling is useful because it is predictable.** Fit a power law on small runs, then commit the budget once. How that budget splits between parameters and data is an empirical question, and the first published answer was revised.
- ▶ **Tokenization is a modelling decision.** It fixes sequence length, cost per language, and which rare strings the model has barely trained on.
- ▶ **Context length is a capacity, and an expensive one.** Retrieval inside the window degrades before the token limit is reached, and the KV cache grows with every token.

- ▶ **Long Context Handling:** Efficient token reuse, sparse attention, recurrence.
- ▶ **Token-Free Models:** Improved byte/character models.
- ▶ **Multimodal LLMs:** Integrating vision, speech, and more.
- ▶ **Smaller, Efficient LLMs:** Distillation, quantization, sparse models.
- ▶ **Open-Weight & Ethical LLMs:** Responsible, accessible models.

These slides have been adapted from:

- ▶ Younes Mourri & Lukasz Kaiser, [Natural Language Processing Specialization. DeepLearning.AI](#)
- ▶ Bhiksha Raj & Rita Singh, [11-785 Introduction to Deep Learning, CMU](#) (Spring 2024), Lecture 19: *Transformers and Newer Architectures*

1. Vaswani et al., “**Attention Is All You Need**”, NeurIPS 2017.
2. Devlin et al., “**BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**”, NAACL-HLT 2019.
3. Radford et al., “**Language Models are Unsupervised Multitask Learners (GPT-2)**”, OpenAI, 2019.
4. Brown et al., “**Language Models are Few-Shot Learners (GPT-3)**”, NeurIPS 2020.
5. Kaplan et al., “**Scaling Laws for Neural Language Models**”, 2020.
6. Hoffmann et al., “**Training Compute-Optimal Large Language Models**” (Chinchilla), NeurIPS 2022.
7. Hestness et al., “**Deep Learning Scaling is Predictable, Empirically**”, 2017.
8. Wei et al., “**Emergent Abilities of Large Language Models**”, TMLR 2022.
9. Wei et al., “**Finetuned Language Models Are Zero-Shot Learners**” (FLAN), ICLR 2022.
10. Touvron et al., “**Llama 2: Open Foundation and Fine-Tuned Chat Models**”, 2023.
11. OpenAI, “**GPT-4 Technical Report**”, 2023.

12. Xue et al., “ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models”, TACL 10, 2022.
13. Clark et al., “CANINE: Pre-training an Efficient Tokenization-Free Encoder for Language Representation”, TACL 10, 2022.
14. Tay et al., “Charformer: Fast Character Transformers via Gradient-based Subword Tokenization”, ICLR 2022.
15. Yu et al., “MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers”, NeurIPS 2023.
16. Pagnoni et al., “Byte Latent Transformer: Patches Scale Better Than Tokens”, 2024.
17. Petrov, La Malfa, Torr, Bibi, “Language Model Tokenizers Introduce Unfairness Between Languages”, NeurIPS 2023.
18. Land & Bartolo, “Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models”, EMNLP 2024.
19. Raschka, “Build a Large Language Model (From Scratch)”, Manning, 2024.
20. Qarah & Alsanoosy, “A Comprehensive Analysis of Various Tokenizers for Arabic Large Language Models”, Applied Sciences 14(13), 2024.
21. Beltagy et al., “Longformer: The Long-Document Transformer”, 2020.
22. Tay et al., “Efficient Transformers: A Survey”, 2020.
23. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, NeurIPS 2022.
24. Anthropic, “Introducing Claude 3.5 Sonnet”, 2024.